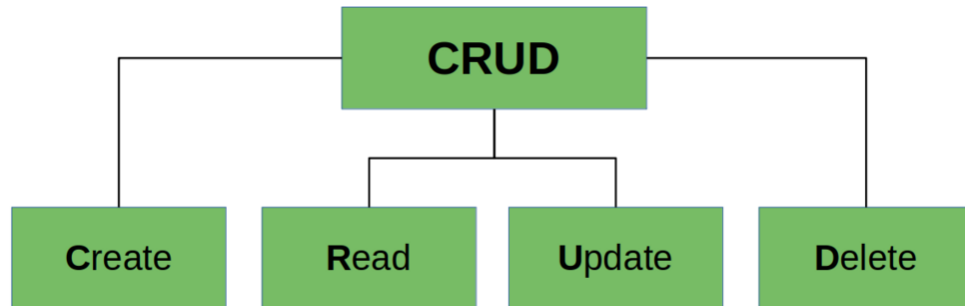


Spring MVC

Spring MVC CRUD :-

- CRUD means Create, Read, Update, and Delete.
- These four operations are the basics of any project.



- Spring MVC is a popular framework for web applications.
- It uses the Model-View-Controller (MVC) architecture.
- The Model handles data.
- The View shows the user interface.
- The Controller manages the business logic.

1. Create – Insert New Data

- Create means adding a new record to the database using a form.
- The controller receives the form data and the DAO inserts it using an SQL INSERT query.

2. Read/Retrieve – Fetch Data

- Read means fetching and displaying existing records from the database.
- The controller calls the DAO to run a SELECT query and sends the results to the view.

3. Update – Modify Data

- Update means editing an existing record and saving the changes.
- The controller processes updated form data, and the DAO runs an UPDATE query.

4. Delete – Remove Data

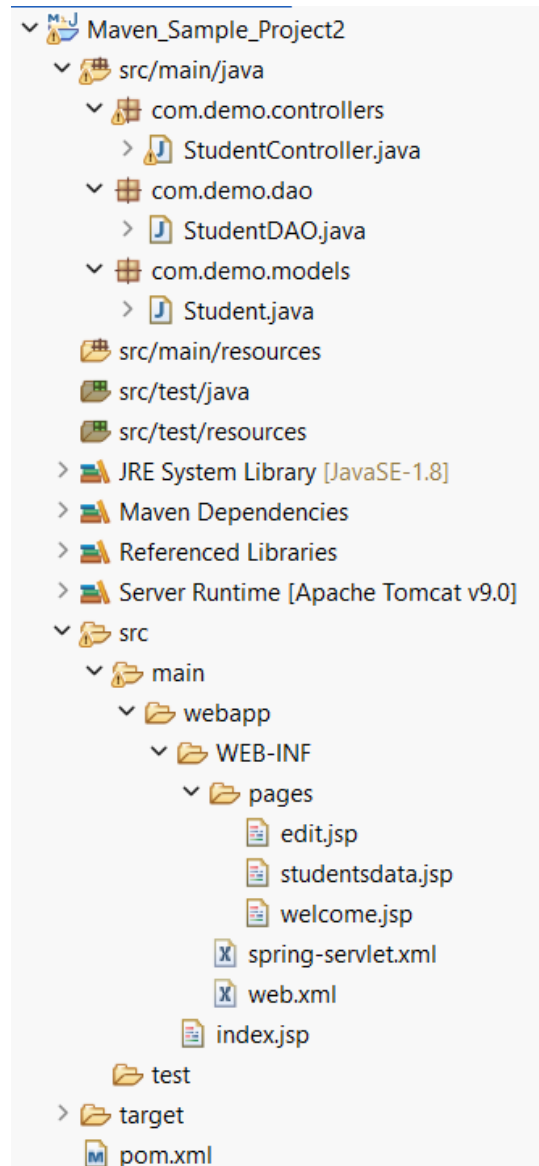
- Delete means removing a record permanently from the database.
- The controller sends the record ID to the DAO, which executes a DELETE query.

CRUD Example of Spring MVC :-

We will build a simple registration form CRUD application focused on the Spring MVC module.

Project Structure -

The project structure for the Spring MVC application is as follows:



Add Dependencies to pom.xml –

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.demo</groupId>
  <artifactId>Maven_Sample_Project2</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.30</version>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>

    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.6.0</version>
    </dependency>

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
      <version>1.2</version>
    </dependency>

  </dependencies>

  <build>
    <finalName>Maven_Sample_Project2</finalName>
  </build>
</project>
```

Create the Model Layer –

```
package com.demo.models;

public class Student {
    private int id;
    private String fullname;
    private String email;
    private String password;
    private String address;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getFullname() {
        return fullname;
    }
    public void setFullname(String fullname) {
        this.fullname = fullname;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

Create the DAO (Data Access Object)/Repository Layer –

```
package com.demo.dao;

import java.util.List;

import org.springframework.jdbc.core.JdbcTemplate;

import com.demo.models.Student;

public class StudentDAO {

    JdbcTemplate template;

    public void setTemplate(JdbcTemplate template) {
        this.template = template;
    }

    public boolean registration(Student s) {
        // String sql="insert into student(full_name,email,password)
        // values('"+s.getFullname()+"','"+s.getEmail()+"','"+s.getPassword()+"','"+s.getAddress()+"')";
        // template.update(sql);

        String sql="insert into student(full_name,email,password,address) values(?,?,?,?)";
        template.update(sql,s.getFullname(),s.getEmail(),s.getPassword(),s.getAddress());
        return true;
    }

    public List<Student> getAllStudents(){
        String sql = "select * from student order by id DESC";

        return template.query(sql,(rs,numberofrows)->{
            Student s=new Student();
            s.setId(rs.getInt("id"));
            s.setFullname(rs.getString("full_name"));
            s.setEmail(rs.getString("email"));
            s.setPassword(rs.getString("password"));
            s.setAddress(rs.getString("address"));
            return s;
        });
    }

    public void delete(int id) {
        String sql="delete from student where id=?";
        template.update(sql,id);
    }

    public Student getStudentById(int id) {
        String sql = "select * from student where id=?";

        return template.queryForObject(sql,(rs,numberofrows)->{
            Student s=new Student();
            s.setId(rs.getInt("id"));
            s.setFullname(rs.getString("full_name"));
            s.setEmail(rs.getString("email"));
            s.setPassword(rs.getString("password"));
            s.setAddress(rs.getString("address"));
            return s;
        },id);
    }

    public void updateStudent(Student s) {
        String sql="update student set full_name=?,email=?,password=?,address=? where id=?";
        template.update(sql,s.getFullname(),s.getEmail(),s.getPassword(),s.getAddress(),s.getId());
    }
}
```

Create the Controller Layer –

```
package com.demo.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import com.demo.dao.StudentDAO;
import com.demo.models.Student;

@Controller
public class StudentController {

    // @PostMapping("/add")
    // public String addition(@RequestParam int num1, @RequestParam int num2, Model m)
    // {
    //     int result=num1+num2;
    //     System.out.println("Addition= "+result);
    //     m.addAttribute("result",result);
    //     return "welcome"; //welcome.jsp
    // }

    // @GetMapping("/welcome")
    // public String wel() {
    //     return "welcome"; //welcome.jsp
    // }

    @Autowired
    StudentDAO sdao;

    @PostMapping("/save")
    public String save(@ModelAttribute Student s) {
        boolean result=sdao.registration(s);
        if(result) {
            System.out.println("Registration Successfully Completed");
            return "redirect:/studentsdata";
        }else {
            System.out.println("Fail");
            return "redirect:/";
        }
    }

    @GetMapping("/studentsdata")
    public String studentsdata(Model m) {
        List<Student> list=sdao.getallstudents();
        m.addAttribute("list",list);
        return "studentsdata"; //studentsdata.jsp
    }

    @GetMapping("/deletestudent/{id}")
    public String deletestudent(@PathVariable int id) {
        sdao.delete(id);
        return "redirect:/studentsdata";
    }

    @GetMapping("/profileedit/{id}")
    public String profileedit(@PathVariable int id, Model m) {
        Student s= sdao.getStudentById(id);
        m.addAttribute("student",s);
        return "edit"; //edit.jsp
    }

    @PostMapping("/updatestudent")
    public String updatestudent(@ModelAttribute Student s) {
        sdao.updatestudent(s);
        return "redirect:/studentsdata";
    }
}
```

The key annotation include:

1. **@Controller**: This annotation marks that the particular class serves the role of a controller.
2. **@GetMapping**: This annotation marks the mapping of HTTP GET requests onto specific handler methods.
3. **@PostMapping**: This annotation marks the mapping of HTTP POST requests onto specific method handlers.
4. **@PathVariable**: Binds a method parameter to a URI template variable.
5. **@ModelAttribute**: Binds form data to a model object.

Studentdata.jsp Page –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Students Data</h1>

    <table border="1">
        <tr>
            <th>Id</th>
            <th>Fullname</th>
            <th>Email</th>
            <th>Password</th>
            <th>Address</th>
            <th></th>
        </tr>

        <c:forEach var="s" items="${list}" varStatus="status">
            <tr>
                <td>${status.index + 1}</td>
                <td>${s.fullname}</td>
                <td>${s.email}</td>
                <td>${s.password}</td>
                <td>${s.address}</td>

                <td>
                    <a href="deletestudent/${s.id}">Delete</a>
                    <a href="profileedit/${s.id}">Profile Edit</a>
                </td>
            </tr>
        </c:forEach>

    </table>
</body>
</html>
```

index.jsp Page –

```
<html>
<body>

    <!-- <form action="add" method="post">
        <input type="text" name="num1" placeholder="Enter number1"><br><br>
        <input type="text" name="num2" placeholder="Enter number2"><br><br>

        <button type="submit">Submit</button>
    </form> -->

    <form action="save" method="post">
        <input type="text" name="fullname" placeholder="Enter fullname"><br><br>
        <input type="email" name="email" placeholder="Enter email"><br><br>
        <input type="password" name="password" placeholder="Enter password"><br><br>
        <textarea name="address" placeholder="Enter address"></textarea><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

edit.jsp page –

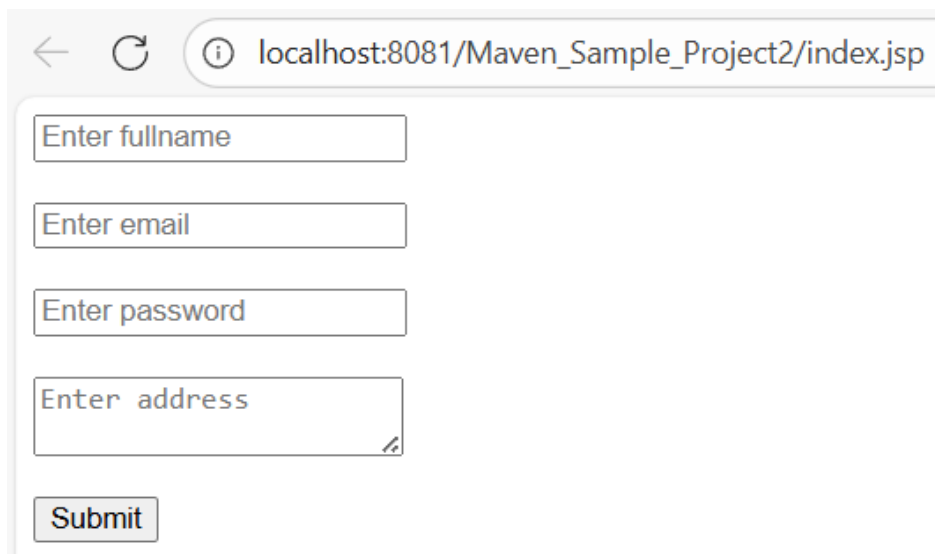
```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

    <form action="../updatestudent" method="post">
        <input type="hidden" name="id" value="${student.id}">
        <input type="text" name="fullname" placeholder="Enter fullname" value= "${student.fullname}"><br><br>
        <input type="email" name="email" placeholder="Enter email" value= "${student.email}"><br><br>
        <input type="password" name="password" placeholder="Enter password" value= "${student.password}"><br><br>
        <textarea name="address" placeholder="Enter address">${student.address}</textarea><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

Test the CRUD Operations –



localhost:8081/Maven_Sample_Project2/index.jsp

Enter fullname

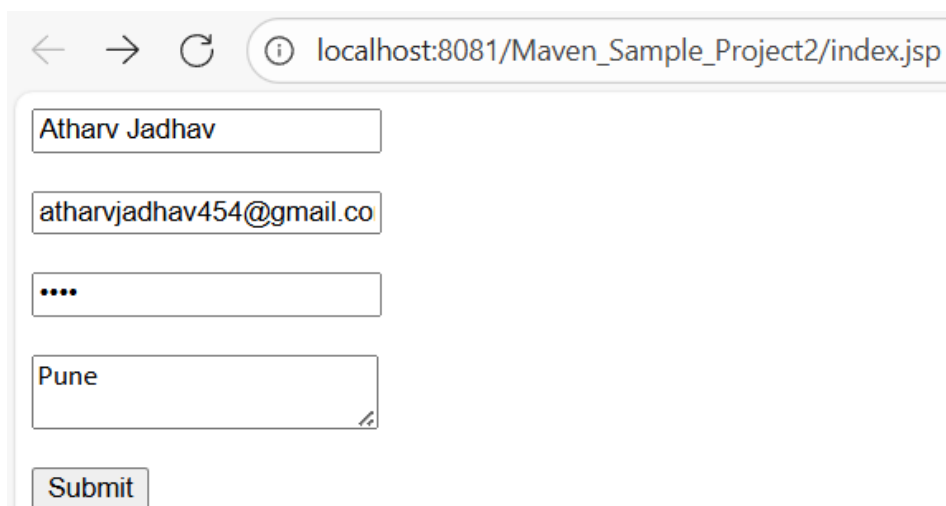
Enter email

Enter password

Enter address

Submit

Creating Student Data -



localhost:8081/Maven_Sample_Project2/index.jsp

Atharv Jadhav

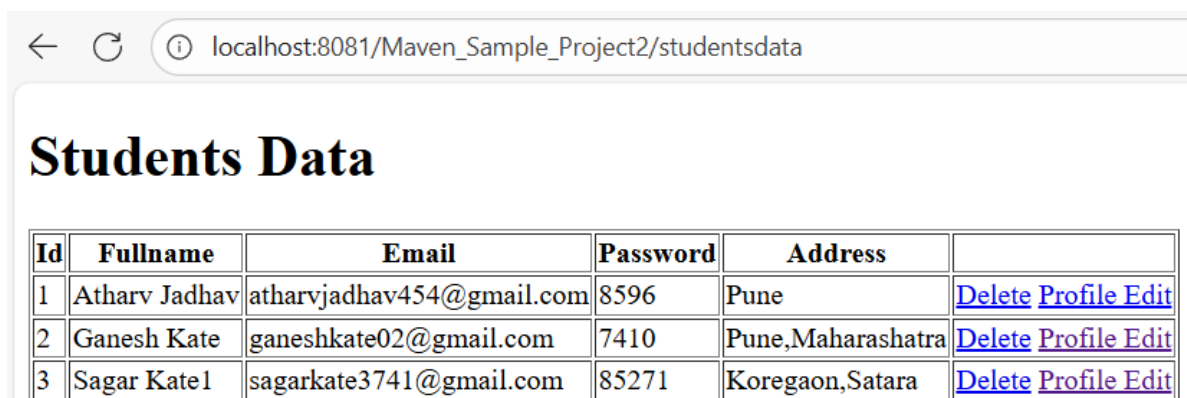
atharvjadhav454@gmail.co

....

Pune

Submit

Reading Student Data -



localhost:8081/Maven_Sample_Project2/studentsdata

Students Data

Id	Fullname	Email	Password	Address	
1	Atharv Jadhav	atharvjadhav454@gmail.com	8596	Pune	Delete Profile Edit
2	Ganesh Kate	ganeshkate02@gmail.com	7410	Pune,Maharashtra	Delete Profile Edit
3	Sagar Kate1	sagarkate3741@gmail.com	85271	Koregaon,Satara	Delete Profile Edit

Deleting Student Data -

← ↻ ⓘ localhost:8081/Maven_Sample_Project2/studentsdata

Students Data

Id	Fullname	Email	Password	Address	
1	Atharv Jadhav	atharvjadhav454@gmail.com	8596	Pune	Delete Profile Edit
2	Sagar Kate1	sagarkate3741@gmail.com	85271	Koregaon,Satara	Delete Profile Edit

Editing Student Data -

← ↻ ⓘ localhost:8081/Maven_Sample_Project2/profileedit/16

Updating Student Data -

← ↻ ⓘ localhost:8081/Maven_Sample_Project2/profileedit/16

Student Data updated successfully:

   localhost:8081/Maven_Sample_Project2/studentsdata					
Students Data					
Id	Fullname	Email	Password	Address	
1	Atharv Jadhav1	atharvjadhav454@gmail.com	85961	Pune1	Delete Profile Edit
2	Sagar Kate1	sagarkate3741@gmail.com	85271	Koregaon,Satara	Delete Profile Edit